Inheritance Exercise

Exercise 1: Person and Student

Create a superclass named Person with two attributes: name and age. Create a subclass named Student with an additional attribute studentId.

Requirements:

1. Create a parameterized constructor in Person.
2. Use super(name, age) in the Student constructor.
3. Create a method displayInfo() in Person that prints the name and age.
4. Override displayInfo() in Student to print the name, age, and student ID.
5. Create two Student objects and display their information.

=

Exercise 2: Vehicle and Car

Create a superclass Vehicle containing brand and speed. Create a subclass Car with an additional attribute numberOfDoors.

Requirements:

1. Initialize brand and speed through the superclass constructor.
2. Call the superclass constructor using super().
3. Define a method startEngine() in Vehicle that prints "Vehicle engine starting".
4. Override startEngine() in Car to print "Car engine starting with key".
5. Create a Car object and call startEngine().

Challenge: Inside the overridden method, call super.startEngine() before printing the car-specific message.

Exercise 3: Animal and Dog

Create a superclass Animal with attributes name and weight. Create a subclass Dog with an additional attribute breed.

Requirements:

1. Write parameterized constructors for both classes.
2. Initialize inherited data using super().

3. Create a method makeSound() in Animal that prints "Animal makes a sound".
4. Override makeSound() in Dog to print "Dog barks: Woof Woof!".
5. Create two Dog objects with different values.
6. Display their information and sounds.

Exercise 4: Employee Salary Management System

Create a superclass Employee containing name, employeeId, and basicSalary.

Create a subclass Manager containing an additional attribute bonus.

Requirements:

1. Initialize all employee information using constructors.
2. Use super() inside the Manager constructor.
3. Define calculateSalary() in Employee that returns basicSalary.
4. Override calculateSalary() in Manager so it returns basicSalary + bonus.
5. Override displayDetails() to print all relevant information.
6. Create objects of Employee and Manager and compare their salaries.

Sample data:

Attribute	**Employee**	**Manager**
Name	Rahim	Karim
ID	101	102
Basic salary	30,000	50,000
Bonus	—	15,000

Expected salary: Employee = 30,000, Manager = 65,000.

Exercise 5: Bank Account System

Create a superclass BankAccount with accountNumber, holderName, and balance.

Create two subclasses:

- SavingsAccount with interestRate.

- CurrentAccount with serviceCharge.

Requirements:

1. All classes must contain parameterized constructors.
2. Both subclasses must call the parent constructor using super().
3. Define calculateYearEndBalance() in BankAccount, returning the balance.
4. Override this method in SavingsAccount to add annual interest.
5. Override it in CurrentAccount to subtract the service charge.
6. Display the final balance for each account.

Example: For an initial balance of 10,000, an interest rate of 5%, and a service charge of 200:

Savings Account Final Balance: 10500.0

Current Account Final Balance: 9800.0

Exercise 6: Shape Area Calculator

Create a superclass Shape with an attribute color.

Create two subclasses: Rectangle and Circle.

Requirements:

1. Define a parameterized constructor in Shape.
2. Call the superclass constructor from both subclasses.
3. Define calculateArea() in Shape, returning 0.0.
4. Override calculateArea() in Rectangle using length * width.
5. Override calculateArea() in Circle using Math.PI * radius * radius.
6. Override displayInfo() to display color, shape type, and area.

Challenge: Store Rectangle and Circle objects inside a Shape[] array and calculate their areas using a loop.

Exercise 7: Online Shopping Product System

Create a superclass Product containing productName and price.

Create two subclasses:

- Electronics with warrantyYears.
- Clothing with discountPercentage.

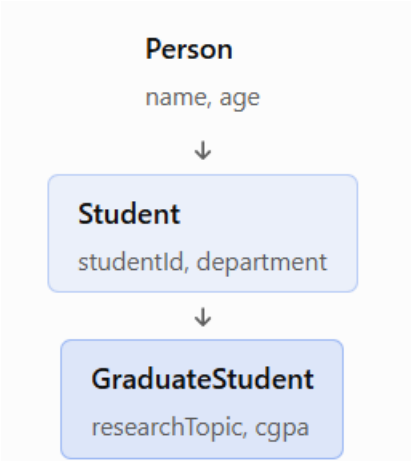
Requirements:

1. Use constructors to initialize all attributes.
2. Call the superclass constructor using super().
3. Define a method getFinalPrice() in the superclass that returns the original price.
4. Override getFinalPrice() in Electronics to add a 10% service charge.
5. Override getFinalPrice() in Clothing to apply the given discount.
6. Print the product details and final price.

Challenge: Create an array of type Product containing both electronics and clothing objects.

Exercise 8: University Management System — Multilevel Inheritance

Design the following inheritance hierarchy:



Requirements:

1. Create a parameterized constructor in each class.
2. Use super() at each inheritance level.
3. Define displayInfo() in Person.
4. Override displayInfo() in Student and GraduateStudent.
5. Use super.displayInfo() to reuse the inherited implementation.

6. Create a GraduateStudent object and display all its information.

Challenge: Print a message from every constructor to demonstrate constructor execution order.

Exercise 9: Transport Fare Calculation System

Create a superclass Transport containing vehicleNumber, distance, and baseFare.

Create three subclasses: Bus, Taxi, and Train.

Requirements:

1. Define parameterized constructors for all classes.
2. Each subclass must use super() to initialize common fields.
3. Define calculateFare() in Transport returning baseFare.
4. Override this method according to the following rules.

Transport	**Fare formula**
Bus	baseFare + distance × 5
Taxi	baseFare + distance × 20
Train	baseFare + distance × 3

5. Store different transport objects inside a Transport[] array.
6. Use a loop to display their calculated fares.

Challenge: Create an overridden displayDetails() method that calls calculateFare().

Exercise 10: Hospital Billing System

Create a superclass Patient containing patientId, name, and consultationFee.

Create two subclasses: InPatient and OutPatient.

Additional attributes:

- InPatient: daysAdmitted, dailyRoomCharge
- OutPatient: testCharge

Requirements:

1. Create constructors for all three classes.
2. Both subclasses must invoke the superclass constructor using super().
3. Define calculateBill() in Patient, returning the consultation fee.
4. Override calculateBill() in InPatient to add room charges.
5. Override calculateBill() in OutPatient to add test charges.
6. Override displayBill() in the subclasses, reusing the superclass method where appropriate.
7. Create several patient objects and display their bills.
8. Demonstrate runtime polymorphism using an array of Patient references.

Challenge: Identify which overridden method executes when a parent reference points to a child object.